

Worksheet SOLUTIONS: Knowledge Representation & Ontologies

Foundations of Artificial Intelligence – SS 2026

University of Bremen, Institute for Artificial Intelligence

Part 1 – Conceptual Questions

1.1 Three components of a DL knowledge base:

- **TBox (Terminological Box)** — the ontology layer. Defines the concept hierarchy: what classes exist, how they relate to each other, and what properties they have. This is intensional/schema knowledge (e.g. `Cat  $\sqsubseteq$  Mammal`).
- **ABox (Assertional Box)** — the instance/data layer. Describes concrete individuals and which classes/roles they belong to (e.g. `Cat(kittypillar)` , `owns(ben, rex)`).
- **RBox (Relational Box)** — describes the hierarchy and constraints on roles/relations themselves (e.g. role inclusion `dreams  $\sqsubseteq$  sleeps` , disjointness `Dis(sleeps, runs)`).

1.2 Closed vs. Open World:

Question	Closed-World	Open-World
Does Ben own a dog?	Yes — Rex is stated to be a dog and Ben owns Rex	Yes — same reasoning applies
Does Ben own a cat?	No — not stated, therefore false	Unknown — Ben could own a cat not mentioned
Is Rex a cat?	No — Rex is stated to be a dog, not a cat	Unknown — Rex could also be a cat (no UNA)

The key difference: under CWA, anything not explicitly stated is assumed false. Under OWA (used in DL/ontologies), absence of information means unknown.

1.3 Ontology vs. Knowledge Graph:

An **ontology** corresponds to the **TBox + RBox** — it defines the vocabulary, class hierarchy, and role structure. It captures the *meaning* of concepts but says nothing about specific individuals.

A **Knowledge Graph** is the combination of an ontology (T+R) with an **ABox** of individuals, structured as a graph of subject–relation–object triples. It extends the ontology with actual instance data. A KG also typically acquires data from multiple web sources, whereas a pure ontology is a standalone schema.

1.4 ALC constructs:

Construct	Meaning
$\top$	Everything / the universal concept (all individuals)

Construct	Meaning
$\bot$	Nothing / the empty concept (no individuals)
$C \sqcap D$	Conjunction — individuals that are in both C and D
$C \sqcup D$	Disjunction — individuals that are in C or D (or both)
$\exists r.C$	Existential restriction — individuals that have at least one r-relation to some C
$\forall r.C$	Universal restriction — individuals all of whose r-successors are in C
$C \sqsubseteq D$	Inclusion — every C is also a D (C is a subclass of D)

1.5 TBox / ABox / RBox:

Statement	Answer
<code>Cat $\sqsubseteq$ Mammal</code>	TBox
<code>Mother(julia)</code>	ABox
<code>dreams $\sqsubseteq$ sleeps</code>	RBox
<code>HappyCatOwner $\sqsubseteq$ $\exists$owns.Cat</code>	TBox
<code>ParentOf(julia, max)</code>	ABox
<code>Dis(sleeps, runs)</code>	RBox

Part 2 – Reading DL Axioms

(a) `Lecturer  $\sqsubseteq$   $\exists$ teaches.Lecture`

Every lecturer teaches at least one lecture.

(b) `Mother  $\equiv$  Female  $\sqcap$  Parent`

Being a mother is exactly the same as being both female and a parent.

(c) `Male  $\sqcap$  Female  $\sqsubseteq \bot$`

Nothing can be both male and female at the same time (the intersection is empty).

(d) `HappyCatOwner  $\sqsubseteq$   $\exists$ owns.Cat  $\sqcap$   $\forall$ caresFor.Healthy`

Every happy cat owner owns at least one cat, and everything they care for is healthy.

(e) `owns  $\sqsubseteq$  caresFor` (RBox)

If someone owns something, they also care for it (owning implies caring for).

Part 3 – Writing DL Axioms

(a) `Student  $\sqsubseteq$  Person`

(b) `Vegetarian  $\equiv$  Person  $\sqcap$   $\forall$ eats.PlantBasedFood`

(c) `Alive ⊓ Dead ⊆ ⊥`

(d) `Robot ⊆ ∃hasSensor.Sensor`

(e) `perceives ⊆ isAwareOf (RBox)`

(f)

```
Robot(tiago)           ← ABox: tiago is a Robot
locatedIn(tiago, kitchen) ← ABox: role assertion
Kitchen(kitchen)       ← ABox: kitchen is a Kitchen
```

Part 4 – Ontology in Python (Solutions)

See the notebook `ontology_exercises.ipynb` for the complete runnable solution.

Key points:

4.1 TBox design — the reasoner uses `SubClassOf` axioms and restrictions like `some` ($\exists$) and `only` ($\forall$) to define class semantics. Example:

```
class HappyCatOwner(Thing): pass
HappyCatOwner.equivalent_to = [
    Person & owns.some(Cat) & caresFor.only(Healthy)
]
```

4.2 ABox — asserting `Robot(tiago)` and `locatedIn(tiago, kitchen)` is enough; the reasoner fills in what can be inferred from the TBox (e.g. that tiago is also a `PhysicalObject` if `Robot ⊆ PhysicalObject`).

4.3 Reasoning results — after running `sync_reasoner_pellet()`, `tiago.is_a` will include inferred superclasses. The reasoner applies all applicable GCIs transitively.

4.4 Inconsistency — making an individual both `Alive` and `Dead` violates `Alive ⊓ Dead ⊆ ⊥`. The reasoner will classify the ontology as **inconsistent** and raise an `OwlReadyInconsistentOntologyError`.

End of solutions. 🎓